

Lab 3: Auditing Your ETL Pipeline · Tonight in 20 Steps

OPIM 5512 · Module 3 · Solo · Colab · your own Google Cloud bucket

The one-sentence version: before you trust a model, check the data your pipeline is feeding it. You load your own cars, read the real ads, grade what the regex robot extracted, and fix the worst field with one regular expression.

Before we start

- [] **5 green checks** in your fork's **Actions** tab (Final GCP Guide F26, Steps 0 to 10)
- [] Your bucket has `structured/datasets/listings_master.csv` (Force run scraper → extractor → materialize-master)
- [] The lab notebook is open in Colab: HuskyCT → In-Class Labs → **Lab 3: Auditing Your ETL Pipeline**

⚠ The 3 things that trip everyone up (read this first)

1. **The Colab sign-in pop-up: pick the Gmail you used for Google Cloud**, not necessarily your UConn account. Wrong account = "permission denied" or an empty bucket.
2. **listings_master.csv not found?** In Cloud Scheduler, **tick the box first**, then Force run the **extractor**, wait for **Success**, then **materialize-master**. One at a time.
3. **Save your work:** the notebook opens from GitHub, so first do **File** → **Save a copy in Drive**. Otherwise your edits disappear when you close the tab.

Pipeline still not ready after Part 1? Set `USE_BACKUP_DATA = True` in Part 2 and use Dr. Wanik's Hartford cars. You can do the whole lab tonight and fix your pipeline after.

The 20 steps, in order

Part 1 · Get it running (steps 1–5, 30 min)

1. Open the notebook from HuskyCT → **File** → **Save a copy in Drive** (work in the copy).
2. Your fork's **Actions** tab: 5 green checks? If not, work the **Final GCP Guide F26** now. Stuck? Paste the step, the command, and the full error into Claude.
3. **Cloud Scheduler** → tick `craigslist-scraper-hourly` → **Force run** → wait for **Success**.
4. Tick `extractor-per-listing-hourly` → **Force run** → **Success**. Then `materialize-master-hourly` → **Force run** → **Success**.
5. **Cloud Storage** → your bucket → `structured/datasets/listings_master.csv` is there.

Part 2 · Search and see (steps 6–10, 25 min)

6. **2a:** type your `PROJECT_ID` and `BUCKET_NAME` (or set `USE_BACKUP_DATA = True`) → run both cells → pick your Google Cloud Gmail → **Allow**.
7. **2b:** run it → click the **folder icon** on the left → `my_cars` → double-click a `.txt` (the raw ad) and its `.json` (what the robot extracted). Backup data? Open **01** (the robot copied the seller's `Hyunda` typo) and **03** (make `= Contact`, `model = Information`).
8. **2c:** change `SEARCH_WORD` to a brand (`toyota`, `honda`, `ford`) → run → how many of your ads mention it?
9. **2d:** pick **3 cars**. Open each one's link in a new tab, next to its `.txt` file.
10. Write 2–3 sentences: what does the raw text keep from the web page, and what does it lose?

Part 3 · Grade the robot (steps 11–14, 30 min)

11. **3a:** run it → copy the printed `truth = { ... }` skeleton into the next cell.
12. Fill in the **truth** for your 5 cars from the ads. **Rules:** numbers without `$` or commas · make and model in lowercase · model = first word · write the **real** make (a `Hyunda` is a `hyundai`).
13. **3b:** run it → the **before** table. Which fields does the robot nail? Which does it miss?
14. **3c:** run both cells → what is the old regex grabbing? (`Contact Information`? A town name?) Then look at the details block: a line that is just the **year**, then **make model** on the next line.

Part 4 · Fix it three ways (steps 15–18, 30 min)

15. **Way 1** (given): read the plain-Python version → run it.
16. **Way 2** (yours): write **one line**, `BETTER_MAKE_MODEL_RE = re.compile(r"...", re.MULTILINE)`. Use the hints (click to open) and **regex101.com** (Python flavor). Run it: you should get something like `('hyunda', 'sonata')`.
17. **Way 3** (stretch): run the typo-fixer → `hyunda` becomes `hyundai`.
18. **4b:** run the re-grade → robot vs way 1 vs **your regex** vs way 3. (*Stretch cell: how many of ALL your ads does your regex miss?*)

Part 5 · Wrap up (steps 19–20, 5 min)

19. Run the **save** cell (your truth table goes to your bucket for **A07**, where Gemini gets graded on the same 5 cars) → answer the **3 short questions**.
20. **File** → **Download** → **Download .ipynb** → HuskyCT → **Submit Lab 3** (due **Thu Oct 8, 11:59 PM**, both campuses).

Definition of done: a truth table for 5 cars · your regex line · the before-vs-after table · 3 short answers · submitted. **Leave your pipeline running.** It adds up to 10 new cars every hour, and your midterm uses them.